

GraphHarvester — Architecture Document

1. Scale Strategy: 500,000+ Records Without Manual Intervention

Horizontal scraper parallelism

Each scraper domain gets its own `asyncio.Semaphore` pool. At scale, these become independent worker pods (e.g. Kubernetes Jobs), one per data source, all writing to a shared message queue (Redis Streams or Kafka). A central coordinator dispatches work and collects progress.

Pagination depth

ArXiv: The Arxiv API returns up to 30,000 results per search query. Running 5–10 distinct queries (by cs.AI, cs.LG, cs.CL, cs.CV, stat.ML) yields 150,000–300,000 unique papers.

Papers with Code: ~100,000 papers indexed. Full pagination is feasible in ~2 hours at 50 items/request with 10 concurrent workers.

YC Companies: ~4,000 companies (full corpus in one render). Supplemented by Crunchbase (via their CSV data export program), AngelList Talent API, and startup directories scraped weekly.

Product Hunt: GraphQL API paginates up to ~50,000 products. Daily delta runs (new posts only) keep the dataset current with minimal API calls.

Scheduling

A cron-based scheduler (Apache Airflow or GitHub Actions workflows) runs:

Bulk scrapers (ArXiv, PwC, YC): weekly — these sources update slowly.

Fresh scrapers (news, jobs): every 4 hours — freshness window is 24hrs, so 6 runs/day guarantees no articles are missed.

Deduplication at scale

URL-based deduplication uses a Redis SET (`SADD url_seen` — O(1) per check). At 500K records this fits in <50 MB of RAM. For cross-source entity deduplication, a probabilistic Bloom filter (e.g. RedisBloom) can track 10M URLs with ~12 MB and <1% false positive rate.

2. Handling 413 Payload Errors and 429 Rate Limits at Scale

413 Payload Too Large

The LLM extractor's `_smart_truncate()` function caps input at ~6,000 tokens (~24,000 chars) before every API call. For larger documents:

Chunking: Split into 3,000-token overlapping chunks, run extraction on each, merge results (deduplicate entities by name + type).

Prioritised truncation: Always keep the first 500 words (title/author/price data density is highest here) + signal paragraphs.

Retry with smaller payload: If a 413 is still received (e.g. from a proxy misconfiguration), halve the chunk size and retry once.

429 Rate Limits

Three-layer defence:

LayerMechanismScope Per-call retry`tenacity` exponential backoff (2s→60s + jitter), max 4 attemptsSingle provider Provider fallbackGemini → Groq → DeepSeek — never retry a 429 provider more than 4 times before moving onPer extraction task Global rate limiter`RateLimiter` token-bucket, configured per domain (e.g. 5 req/s for GitHub API)All concurrent workers At distributed scale (100+ workers), a **Redis token bucket** (via `redis-py` + Lua scripting) replaces the in-process `asyncio.Semaphore` to enforce per-domain rate limits across nodes.

For LLM APIs specifically, each provider's rate limit is tracked in Redis with a sliding window counter. When a provider's counter is near its limit, new tasks are routed to the next provider proactively — avoiding 429s rather than recovering from them.

3. Guaranteeing No Article Is Processed Twice Across Distributed Nodes

Seen-URL cache (current implementation)

`logs/seen_urls.json` — single-node, file-based. Works for development.

Distributed deduplication (production design)

Primary mechanism: Redis SET with URL hashing

```
# On each worker before processing:
url_hash = sha256(url)
is_new = REDIS.setnx(f"seen:{url_hash}", iso_timestamp) # atomic, returns 1 only once
if not is_new:
    skip()
```

`SETNX` is atomic — even if 50 workers race on the same URL simultaneously, exactly one will get `1` (new) and proceed; the rest get `0` and skip.

TTL for news/jobs (time-bounded freshness)

News and job URLs are set with a 30-day TTL (`SETEX`). This ensures the seen-set doesn't grow unbounded for content that cycles (e.g. recurring "hiring" posts).

Message queue deduplication

Each scraper publishes `{url, record_type, raw_text}` to a Kafka topic with the URL as the partition key. Kafka's log-compaction guarantees that only the latest record per key is retained, eliminating duplicates at the storage layer before LLM extraction.

Idempotent Sheets write

Each record is keyed by `(source_url, recordType)` before writing to Sheets. The writer checks the last N rows of the target tab for existing URLs (using a Sheets `MATCH` formula or a local in-memory set loaded on startup) and skips duplicates.

4. Primary Database + Vector/Graph Storage

Primary database: `**PostgreSQL**`

Rationale:

JSONB column for the canonical schema (`content` field) — queryable without a fixed schema.

`source_url` as a unique index — deduplication at the DB level as a final safety net.

Full-text search via `tsvector` for title/description search without an additional search service.

Row-level timestamps enable incremental processing (process only rows newer than last run).

Mature ecosystem, ACID guarantees, managed versions on every cloud (RDS, Cloud SQL, Neon).

Schema sketch:

```
CREATE TABLE records (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  record_type TEXT NOT NULL,      -- STARTUP | PRODUCT | RESEARCH_PAPER | JOB | NEWS
  source_url  TEXT UNIQUE NOT NULL,
  content     JSONB NOT NULL,
  collected_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  schema_ver  TEXT NOT NULL DEFAULT '1.0'
);
CREATE INDEX records_type_idx ON records(record_type);
CREATE INDEX records_collected_idx ON records(collected_at);
```

Vector storage: `**pgvector**` (extension on PostgreSQL)

Rationale for keeping it in Postgres rather than a separate vector DB:

Embedding the `content.description` / `content.abstract` as a `vector(1536)` column allows semantic similarity search without introducing Pinecone/Weaviate operational overhead.

For <5M records, pgvector with an IVFFlat index (nlist=100) gives sub-100ms similarity queries.

At >10M records or <10ms latency requirements, migrate to **Qdrant** (Rust, single binary, excellent filter-then-vector performance).

Graph storage: **Neo4j**

Used for relationship mapping between entities (startups ↔ investors ↔ products ↔ founders ↔ papers):

Node types: Startup, Product, Person, Paper, Institution

Edge types: FOUNDED_BY, ACQUIRED, CITED_IN, BUILT_ON, WORKS_AT

Populated by the entity resolver: when two records reference the same canonical entity, edges are created between them.

Cypher queries enable traversals like "find all papers cited by OpenAI researchers that led to a product launched in the last 6 months."

Integration flow

```
Scraper → PostgreSQL (raw + structured)
      ↓
Embedding worker → pgvector column (semantic search)
      ↓
Graph builder → Neo4j (entity relationships)
      ↓
Google Sheets (human-readable exports, ops team)
```

*Export this file to PDF (3 pages max). Recommended: `pandoc architecture.md -o architecture.pdf --pdf-engine=xelatex` *